Guia de Estudio Integral

Proyecto 3: Tablas de paginas en accion (xv6)

Sistemas Operativos 2026-2 - Universidad EAFIT

Base: xv6-riscv, commit 06aad25

Documento de estudio y preparacion para la sustentacion. Cubre: contexto desde cero, la teoria de paginacion Sv39, la logica de desarrollo, el analisis exhaustivo de cada archivo y funcion (kernel y userspace), y un guion para la presentacion. Esta ordenado de lo mas simple a lo mas tecnico.

Contenido:

PARTE 0 - El problema desde cero (nivel basico)

0.1 Que es un sistema operativo y que es xv6

Un sistema operativo (SO) es el programa maestro que administra el computador: reparte la memoria entre los programas, decide cual se ejecuta en cada momento, controla el disco, el teclado, etc. Sin el, los programas no podrian convivir.

xv6 es un sistema operativo pequenito y educativo, hecho en el MIT, que imita a Unix. Es ideal para aprender porque tiene pocas lineas y se puede entender completo. Corre sobre un procesador RISC-V (una arquitectura moderna y abierta) que simulamos con un programa llamado QEMU. QEMU es como un 'computador virtual' dentro de tu computador: finge ser un equipo RISC-V para que xv6 piense que corre en hardware real.

0.2 El concepto central: la memoria virtual

Esta es la idea mas importante de todo el proyecto. Imagina un edificio de apartamentos. Cada inquilino (cada programa) cree que su apartamento es el 'numero 1' y que el edificio es suyo. Pero en realidad hay muchos inquilinos compartiendo el mismo edificio (la memoria RAM). Para que la ilusion funcione, hace falta un conserje que traduzca: 'cuando el inquilino A dice apartamento 1, en realidad es el 305; cuando el inquilino B dice apartamento 1, es el 712'.

Ese conserje es la TABLA DE PAGINAS. Traduce las direcciones que ve el programa (direcciones VIRTUALES) a las direcciones reales en la RAM (direcciones FISICAS).

- Direccion virtual (VA): la que usa el programa. Es la 'mentira util'.
- Direccion fisica (PA): donde esta de verdad el dato en la RAM.
- Pagina: la memoria no se traduce byte por byte, sino en bloques de 4096 bytes (4 KB) llamados paginas. Es como manejar el edificio por pisos, no por ladrillos.

Gracias a esto, cada programa esta AISLADO: no puede leer ni danar la memoria de otro, porque su tabla de paginas solo traduce SUS propias direcciones. La memoria virtual es, a la vez, una comodidad (cada quien cree tener todo) y una proteccion (nadie pisa a nadie).

0.3 Que nos pide exactamente el Proyecto 3

El proyecto nos hace TOCAR DIRECTAMENTE esa tabla de paginas con tres ejercicios independientes. Cada uno ensena un concepto distinto de sistemas operativos:

- Parte A - Pagina compartida y ugetpid(): lograr que un programa sepa su numero de proceso (pid) simplemente LEYENDO una posicion de memoria, sin tener que pedirselo al kernel. Es una optimizacion de velocidad.
- Parte B - vmprint(): construir una herramienta que IMPRIME la tabla de paginas del proceso, mostrando sus tres niveles. Es como abrir el cuaderno del conserje y leer todas sus anotaciones.
- Parte C - pgaccess(): DETECTAR que paginas uso un proceso recientemente, consultando un bit que el hardware enciende solo cada vez que una pagina se lee o escribe. Es la base de los algoritmos que deciden que sacar de memoria cuando esta se llena.

0.4 Dos conceptos que hay que dominar antes del codigo

(a) Llamada al sistema (syscall) y por que es 'cara'

Cuando ur
saber su
usuario' a
del kernel
Parte A b
programa

(b) Los permisos de una PTE (Page Table Entry)

Cada entra
unas BAN

- PTE_V (Valid): la entrada existe y es valida.
- PTE_R (Read): se puede leer.
- PTE_W (Write): se puede escribir.
- PTE_X (eXecute): se puede ejecutar como codigo.
- PTE_U (User): el programa de usuario puede acceder. Sin este bit, solo el kernel.
- PTE_A (Accessed): el HARDWARE lo enciende automaticamente cuando la pagina se lee o escribe. Es la estrella de la Parte C.

Idea clave para toda la Parte A:

Si mapear
escribirla.
protegemo

PARTE 1 - Teoria: paginacion RISC-V Sv39

Esta parte es la 'matematica' del proyecto. Es indispensable para entender las Partes B y C. Tomala con calma; con un ejemplo se entiende bien.

1.1 Direcciones de 39 bits divididas en 3 niveles

xv6 usa el esquema Sv39 de RISC-V. 'Sv39' significa que las direcciones virtuales utiles tienen 39 bits. Esos 39 bits NO se usan de corrido: se parten en cuatro campos, asi:

```
Direccion virtual (39 bits):
```

```
+--------
```

- Los 12 bits de offset = posicion dentro de la pagina. 2^12 = 4096 = 4 KB (por eso las paginas son de 4 KB).
- Cada indice de 9 bits elige una entre 512 entradas (2^9 = 512) dentro de una tabla.
- Hay 3 niveles de tablas encadenadas (L2 -> L1 -> L0). De ahi 'tres niveles'.

1.2 El recorrido de la tabla (walk)

Para traducir una direccion virtual a fisica, el procesador (o la funcion walk() del kernel) hace este recorrido:

- 1. Empieza en la TABLA RAIZ (nivel 2). El indice L2 selecciona una de sus 512 entradas.
- 2. Esa entrada apunta a una tabla de NIVEL 1. El indice L1 selecciona otra entrada.
- 3. Esa apunta a una tabla de NIVEL 0. El indice L0 selecciona la PTE final (la 'hoja').
- 4. La PTE hoja da la direccion fisica de la pagina. Se le suma el offset y listo: tenemos la direccion fisica exacta.

Macros de xv6 (en kernel/riscv.h) que implementan esta matematica:

```
PX(level, va)        -> extrae el indice de 9 bits del nivel indicado
```

```
PXSHIFT(l
```

1.3 Diferencia entre entrada 'interna' y 'hoja'

- Entrada INTERNA (niveles 2 y 1): solo sirve para apuntar a la siguiente tabla. NO tiene encendidos los bits R/W/X.
- Entrada HOJA (normalmente nivel 0): describe una pagina real de datos/codigo. SI tiene algun bit R/W/X encendido.

Regla practica (se usa en vmprint):

Si una PT
bajando).

1.4 El mapa de memoria de un proceso en xv6

De las direcciones bajas a las altas, un proceso tiene:

```
0x0            texto (codigo)          R-X-U
```

Direcciones concretas en este xv6 (utiles para leer la salida de vmprint):

- TRAMPOLINE = 0x3ffffff000
- TRAPFRAME = 0x3fffffe000 (= TRAMPOLINE - 4096)
- USYSCALL   = 0x3fffffd000 (= TRAPFRAME - 4096) <- la que definimos nosotros

La pagina de guarda:

Es una pa
un prograr
que corron

PARTE 2 - Logica de desarrollo (progresion del proyecto)

Este fue el orden logico que seguimos. Entenderlo te permite explicar 'como pensamos' el problema, no solo 'que escribimos'.

Paso 1 - Preparar una base limpia

Clonamos xv6-riscv en el commit exacto que pide la guia (06aad25) y creamos la rama proyecto3. Regla de oro del proyecto: se TRABAJA sobre xv6 completo, pero se ENTREGAN solo los archivos modificados, respetando sus rutas (kernel/..., user/...).

Paso 2 - Entender el codigo base ANTES de tocarlo

Leimos proc.c, vm.c, memlayout.h, riscv.h, sysproc.c, trap.c y exec.c. Esto fue decisivo porque descubrimos dos particularidades de ESTA version del curso:

- Ya trae ASIGNACION PEREZOSA (lazy allocation): existen sbrklazy(), vmfault() e ismapped(). Por eso la guia insiste tanto en 'paginas perezosas no tocadas': una pagina reservada con sbrklazy no tiene memoria fisica hasta que se usa.
- El copyout() tiene 5 argumentos: copyout(pagetable, p->sz, dst, src, len). Distinto del xv6 estandar. Lo tuvimos en cuenta en la Parte C.

Leccion metodologica:

No copiar
decisiones

Paso 3 - Implementar la Parte A (la base de todo)

Definir USYSCALL y struct usyscall; reservar la pagina al crear el proceso; mapearla de solo lectura; liberarla sin fugas; ajustar los limites del heap; y escribir ugetpid() en el espacio de usuario.

Paso 4 - Registrar las syscalls nuevas

vmprint y pgaccess son llamadas al sistema nuevas. Como en el Proyecto 2, cada una necesita: un numero (syscall.h), una entrada en la tabla y su declaracion (syscall.c), un stub en ensamblador (usys.pl) y su firma para el usuario (user.h).

Paso 5 - Implementar la Parte B (vmprint)

Recorrido recursivo de la tabla, siguiendo el modelo de freewalk(), con el formato EXACTO que valida el script del profesor (incluido el espacio inicial en ' ..').

Paso 6 - Implementar la Parte C (pgaccess)

Leer y limpiar el bit PTE_A, con todas las validaciones de argumentos y respetando las paginas perezosas (sin asignarles memoria).

Paso 7 - Verificar exhaustivamente

Compilar sin advertencias; correr pgtbltest (resultado: 11 de 11) y usertests -q (resultado: ALL TESTS PASSED); y revisar automaticamente el formato de vmprint con el script ejecutar_pruebas.py del docente.

PARTE 3 - Analisis profundo del codigo

Para cada archivo: QUE se cambio, POR QUE, a QUE PARTE pertenece, y la logica de sistemas o matematica detras. Primero el kernel, luego el userspace.

3.1 kernel/memlayout.h [Parte A]

Que se anadio:

```
#define USYSCALL  (TRAPFRAME - PGSIZE)
```

Por que y para que:

- USYSCALL es la DIRECCION VIRTUAL de la pagina compartida. Se ubica una pagina (4096 bytes) debajo del TRAPFRAME, tal como exige la guia. Es solo una etiqueta numerica (un #define), no reserva memoria por si misma.
- struct usyscall define el CONTENIDO de esa pagina. Por ahora solo guarda el pid, pero esta pensada para poder crecer (ej: agregar el pid del padre).
- El guardian #ifndef __ASSEMBLER__ ... #endif es una decision obligatoria: memlayout.h tambien lo incluye codigo en ENSAMBLADOR, que no entiende 'struct' de C. Ese guardian hace que la struct solo exista cuando compila C. Sin el, el ensamblador daria error de compilacion.

3.2 kernel/proc.h [Parte A]

```
struct usyscall *usyscall;   // dentro de struct proc
```

- Cada proceso necesita recordar DONDE esta su pagina fisica USYSCALL, para poder llenarla con el pid, mapearla y despues liberarla. Es un puntero a una pagina reservada en el kernel. Se guarda junto a los demas datos del proceso.

3.3 kernel/proc.c [Parte A] - cuatro funciones tocadas

(a) allocproc() - cuando NACE un proceso

```
if ((p->usyscall = (struct usyscall *)kalloc()) == 0) {
```
```
  freepro
```

- kalloc() reserva una pagina fisica de 4 KB y devuelve su direccion (o 0 si no hay memoria). El chequeo con freeproc() evita dejar basura si falla.
- Guardamos el pid DENTRO de esa pagina. Esto es exactamente lo que leera ugetpid() despues. El pid ya fue asignado antes por allocpid().
- Parte A.

(b) proc_pagetable() - arma la tabla de paginas del proceso

```
if (mappages(pagetable, USYSCALL, PGSIZE, (uint64)(p->usyscall),
```

- mappages(tabla, VA, tamano, PA, permisos) crea la traduccion: la direccion virtual USYSCALL apuntara a la pagina fisica p->usyscall.
- DECISION CLAVE: permisos PTE_R | PTE_U, SIN PTE_W. El usuario puede LEER pero NO ESCRIBIR. Esta es la esencia de seguridad de la Parte A.
- Si mappages falla, deshacemos los mapeos previos (trampolin y trapframe) para no dejar la tabla a medias: se evita una fuga de memoria.
- Parte A.

(c) freeproc() - cuando MUERE el proceso

```
if (p->usyscall)
  kfree((
```

- kfree() devuelve la pagina fisica al sistema (libera la memoria). Poner el puntero en 0 evita usarlo por error (puntero colgante).
- Parte A.

(d) proc_freepagetable() - libera la tabla de paginas

```
uvmunmap(pagetable, USYSCALL, 1, 0);
```

- uvmunmap(tabla, VA, n_paginas, do_free) quita el mapeo de USYSCALL.
- DECISION SUTIL (muy preguntable): do_free = 0. Solo quitamos la TRADUCCION, pero NO liberamos la pagina fisica aqui, porque de eso ya se encarga freeproc() con kfree(). Si pusieramos do_free = 1, liberariamos la MISMA pagina DOS VECES (double free), un error grave de memoria. Separar responsabilidades evita el bug.
- Parte A.

(e) growproc() - limite del heap

```
if (sz + n > USYSCALL) {   // antes era TRAPFRAME
  return
```

- El heap crece hacia direcciones mas altas. Antes el tope era TRAPFRAME; ahora es USYSCALL (una pagina mas abajo) para que el heap NUNCA invada la pagina compartida. Requisito 6 de la Parte A.
- Parte A.

3.4 kernel/sysproc.c [Partes A, B y C]

(a) sys_sbrk() - mismo limite para el heap perezoso [Parte A]

```
if (addr + n > USYSCALL)   // antes TRAPFRAME
                                                                    return
```

- Igual que growproc, pero en la rama de asignacion PEREZOSA (sbrklazy). Garantiza que ni siquiera el heap perezoso invada USYSCALL.

(b) sys_vmprint() - handler de la syscall vmprint [Parte B]

```
uint64 sys_vmprint(void) {
                                                                    struct
```

- myproc() devuelve el proceso que esta corriendo ahora.
- Llama a vmprint() con la tabla de ESE proceso y devuelve 0, como pide la guia.
- Parte B.

(c) sys_pgaccess() - handler de pgaccess [Parte C]

```
uint64 sys_pgaccess(void) {
                                                                    uint64
```

Desglose linea por linea:

- argaddr / argint: leen los argumentos que paso el programa de usuario (base, npages, mask). Venian en registros del procesador.
- Las tres validaciones (los if que devuelven -1): base debe estar alineada a pagina (multiplo de 4096); npages entre 1 y 64 (porque la mascara es de 64 bits); y el rango completo debe caber dentro del tamano del proceso (p->sz). Todo esto lo exige la guia.

- walk(tabla, va, 0): baja por los tres niveles hasta la PTE de esa VA. El 0 final (alloc=0) significa 'NO crees tablas nuevas si faltan'. Esto es CRITICO: asi NO asignamos memoria a paginas perezosas. pgaccess observa, no modifica.
- Si la PTE no existe o no es valida -> esa pagina se reporta como 0 (su bit queda apagado). Asi las paginas perezosas no tocadas dan 0.
- *pte & PTE_A: pregunta si el hardware encendio el bit de acceso de esa pagina.
- bitmask |= (1UL << i): operacion de bits. Desplaza un 1 hasta la posicion i y hace OR, encendiendo el bit i de la mascara. Asi codificamos 'la pagina i fue accedida'.
- *pte &= ~PTE_A: apaga el bit PTE_A (AND con el complemento de PTE_A). Lo limpiamos para que la PROXIMA consulta reporte solo accesos NUEVOS. Esta es la base de los algoritmos de reemplazo de paginas.
- copyout(pagetable, p->sz, mask, &bitmask, 8): copia la mascara desde el kernel a la memoria del usuario (direccion mask). Si esa direccion es invalida, falla y devolvemos -1. Nota: en este xv6 copyout tiene 5 argumentos.
- Parte C.

3.5 kernel/vm.c [Parte B] - la funcion vmprint

```
static void
                                                                vmprint_r
```

Explicacion detallada:

- Se recorre igual que freewalk() (la funcion de xv6 que libera tablas): las 512 entradas, de forma RECURSIVA. La recursion es natural porque la tabla es un arbol de 3 niveles.
- LA MATEMATICA DE LA PARTE B: child_va = va | (i << PXSHIFT(level)). Reconstruye la direccion virtual que cubre la entrada i, colocando el indice en la posicion de bits de su nivel. Los niveles superiores ya venian acumulados en 'va'. El offset (12 bits bajos) queda en 0 = INICIO del rango que cubre esa PTE.

- Ejemplo: indice 255 en nivel 2 -> 255 << 30 = 0x3fc0000000. Indice 0 en todos los niveles -> 0x0. Asi salen las direcciones de la salida.
- depth = 2 - level: en la raiz (level 2) imprime 1 grupo ' ..'; en level 1, dos (' .. ..'); en level 0, tres (' .. .. ..'). Coincide EXACTO con el formato que valida el script del profesor (que exige el espacio inicial).
- %p imprime con 0x + 16 digitos hexadecimales (lo hace printptr en printk.c).
- La recursion solo baja en entradas INTERNAS (sin R/W/X) y mientras level > 0. En las hojas se detiene.
- Parte B.

3.6 kernel/riscv.h [Parte C]

```
#define PTE_A (1L << 6) // bit de acceso (bit 6), lo enciende el hardware
```

- Define la bandera del bit de acceso. Segun la especificacion de RISC-V, el bit A esta en la posicion 6 de la PTE. El hardware lo pone en 1 al leer/escribir la pagina; nosotros lo leemos y lo limpiamos. Parte C.

3.7 kernel/defs.h [Parte B]

```
void vmprint(pagetable_t);
```

- Declara el prototipo de vmprint para que sysproc.c pueda llamarla (esta definida en vm.c). defs.h es el 'indice' de funciones del kernel: declara que existen y con que firma.

3.8 Registro de las syscalls nuevas [comun a B y C]

- kernel/syscall.h: numeros nuevos SYS_vmprint (23) y SYS_pgaccess (24). Cada syscall se identifica por un numero.
- kernel/syscall.c: se declara 'extern uint64 sys_vmprint(void);' y la de pgaccess, y se anaden a la tabla syscalls[], que relaciona numero -> funcion que la atiende.
- user/usys.pl: genera el STUB en ensamblador que llama el programa de usuario. El stub coloca el numero en el registro a7 y ejecuta la instruccion 'ecall', que es la que salta al kernel (hace el trap).
- user/user.h: declara las firmas para que los programas de usuario puedan llamar a ugetpid(), vmprint() y pgaccess().

3.9 user/ulib.c [Parte A] - la funcion ugetpid

```
#include "kernel/memlayout.h"   // para USYSCALL y struct usyscall
```

- Interpreta la direccion virtual USYSCALL como un puntero a struct usyscall y lee su campo pid.
- NO hace ninguna llamada al sistema: solo lee memoria. Ese es TODO el punto de la Parte A - evitar el costo del trap al kernel. Funciona porque el kernel ya mapeo esa pagina (de solo lectura) en el espacio del proceso.
- Parte A.

3.10 user/pgtbltest.c - programa de pruebas (del docente)

- No lo escribimos nosotros: lo entrega el profesor. Corre 11 pruebas que verifican las tres partes (ugetpid en procesos e hijos y tras exec, USYSCALL de solo lectura, heap que no invade, sin fugas, vmprint, y pgaccess en sus varios casos). Lo incluimos tal cual para dejar el proyecto listo para compilar.

3.11 Un detalle 'gratis': matar al que escribe en USYSCALL [Parte A]

El requisito A.7 (escribir en USYSCALL debe matar al proceso) NO necesito codigo extra. La razon es elegante:

- La pagina se mapeo SIN PTE_W.
- Si el proceso intenta escribir, el hardware lanza un 'store page fault' (scause 0xf = 15).
- En trap.c, usertrap() intenta resolverlo con vmfault(), pero como la pagina YA esta mapeada (ismapped devuelve 1), vmfault devuelve 0.
- Entonces el flujo cae en el caso por defecto, que llama a setkilled(p). El proceso muere.
- Por eso en pgtbltest ves 'usertrap(): unexpected scause 0xf ... stval=0x3fffffd000' y la prueba igual termina en OK: matar al proceso ERA lo correcto.

PARTE 4 - Guion y recomendaciones para la presentacion

4.1 Estructura sugerida (5 a 8 minutos)

- Contexto (30 s): 'El Proyecto 3 extiende xv6 con tres mecanismos que usan la tabla de paginas: una pagina compartida para acelerar getpid (Parte A), una herramienta para imprimir la tabla (Parte B) y deteccion de paginas accedidas (Parte C).'
- Demo en vivo (lo que el profesor pedira): compilar y correr las dos suites.

```
make qemu
```

```
pgtbltest
```

- Interpretar vmprint en vivo: senalar la linea de USYSCALL (0x3fffffd000, la PTE termina en 13 = V+R+U sin W), TRAPFRAME (...e000) y TRAMPOLINE (...f000). Explicar que la region perezosa solo muestra la pagina que el proceso toco.
- Explicar una decision de diseno por cada parte (ver 4.3).

4.2 Lo mas complejo de abordar (y como lo superamos)

- Entender la version del curso de xv6. No era el xv6 estandar: traia lazy allocation y un copyout de 5 argumentos. Lo resolvimos LEYENDO el codigo base (vm.c, trap.c, exec.c) antes de escribir nada, en vez de pegar soluciones de internet que no encajaban.
- El formato exacto de vmprint. El profesor valida la salida con una expresion regular que exige un ESPACIO inicial (' ..', no '..'). Lo detectamos comparando nuestra salida con el ejemplo oficial y con el script ejecutar_pruebas.py, y ajustamos la indentacion.
- Las pruebas de usertests 'fallando' al principio. lazy_copy y lazy_sbrk fallaban con el usertests.c generico de MIT porque ese archivo no conocia la pagina USYSCALL. Entendimos, leyendo la guia, que el profesor entrega una version ajustada. Con la oficial, paso todo. Esto nos enseno a distinguir 'nuestro codigo' de 'los archivos de prueba'.
- La gestion de memoria sin fugas (lo mas delicado en C). Habia que asegurar que la pagina USYSCALL se libere EXACTAMENTE una vez. La clave fue separar responsabilidades: freeproc hace kfree (libera el fisico) y proc_freepagetable hace uvmunmap con do_free=0 (solo desmapea). Lo verificamos con la prueba sin_fugas_usyscall y con usertests (que detecta fugas de paginas).
- Metodologia: estudio de la especificacion Sv39 y del libro de xv6, iteracion con apoyo de IA para comprender y revisar el codigo, documentacion de cada decision en comentarios, y verificacion empirica en QEMU en cada paso.

4.3 Preguntas probables del profesor (respuestas listas)

- Por que USYSCALL es de solo lectura? Si fuera escribible (PTE_W), el usuario podria falsear su propio pid y, peor, corromper datos de los que depende el kernel. Rompe el aislamiento usuario/kernel.
- Por que do_free=0 al desmapear USYSCALL? Para no liberar dos veces la misma pagina fisica (ya la libera freeproc con kfree). Evita un double free.
- Como se reconstruye la direccion virtual en vmprint? va | (indice << PXSHIFT(level)). Cada nivel aporta 9 bits en su posicion; el offset bajo queda en 0.

- Por que vmprint solo muestra PTEs validas y paginas tocadas? Porque recorremos comprobando PTE_V, y las paginas perezosas no tocadas no tienen PTE valida.
- Por que pgaccess usa walk(..., 0)? Para NO asignar memoria a paginas perezosas: su trabajo es observar el uso de la memoria, no modificarlo.
- Por que pgaccess limpia PTE_A? Para que la siguiente consulta reporte solo accesos nuevos. Es la base de los algoritmos de reemplazo de paginas.
- Que pasa si un programa desborda la pila? Cae en la pagina de guarda (sin PTE_U), el hardware lanza un fallo y el kernel mata al proceso, evitando que corrompa memoria contigua.
- Que otras syscalls se podrian acelerar con la pagina compartida? Las que solo LEEN un dato propio del proceso que cambia poco (como getpid). No las que MODIFICAN estado del kernel (fork, exec, open, write) ni las que leen estado cambiante de otros procesos.

4.4 Checklist final antes de presentar

- make clean && make qemu compila sin advertencias.
- pgtbltest da 11 de 11.
- usertests -q da ALL TESTS PASSED.
- Se interpretar la salida de vmprint (USYSCALL, TRAPFRAME, TRAMPOLINE).
- Se salir de QEMU (Ctrl-a, luego x).
- Puedo explicar una decision de diseno por cada parte (seccion 4.3).
- La entrega esta en el repositorio del grupo, en la carpeta Proyecto3/.

Mensaje final:

Lo mas va
decision. S
proyecto c